

# Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering  
TU Dortmund, Fakultät Informatik

Teil 1.3.1: Nebenläufige Programmierung

v. 07.11.2016

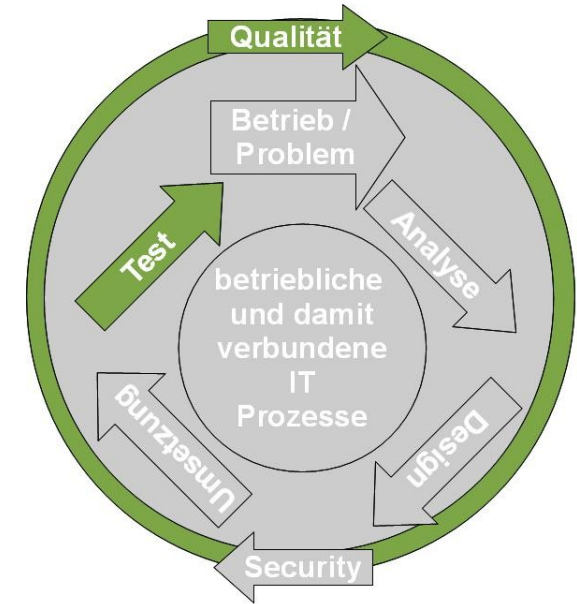
# Einordnung

## Nebenläufige Programmierung

Softwarekonstruktion  
WS 2016/17



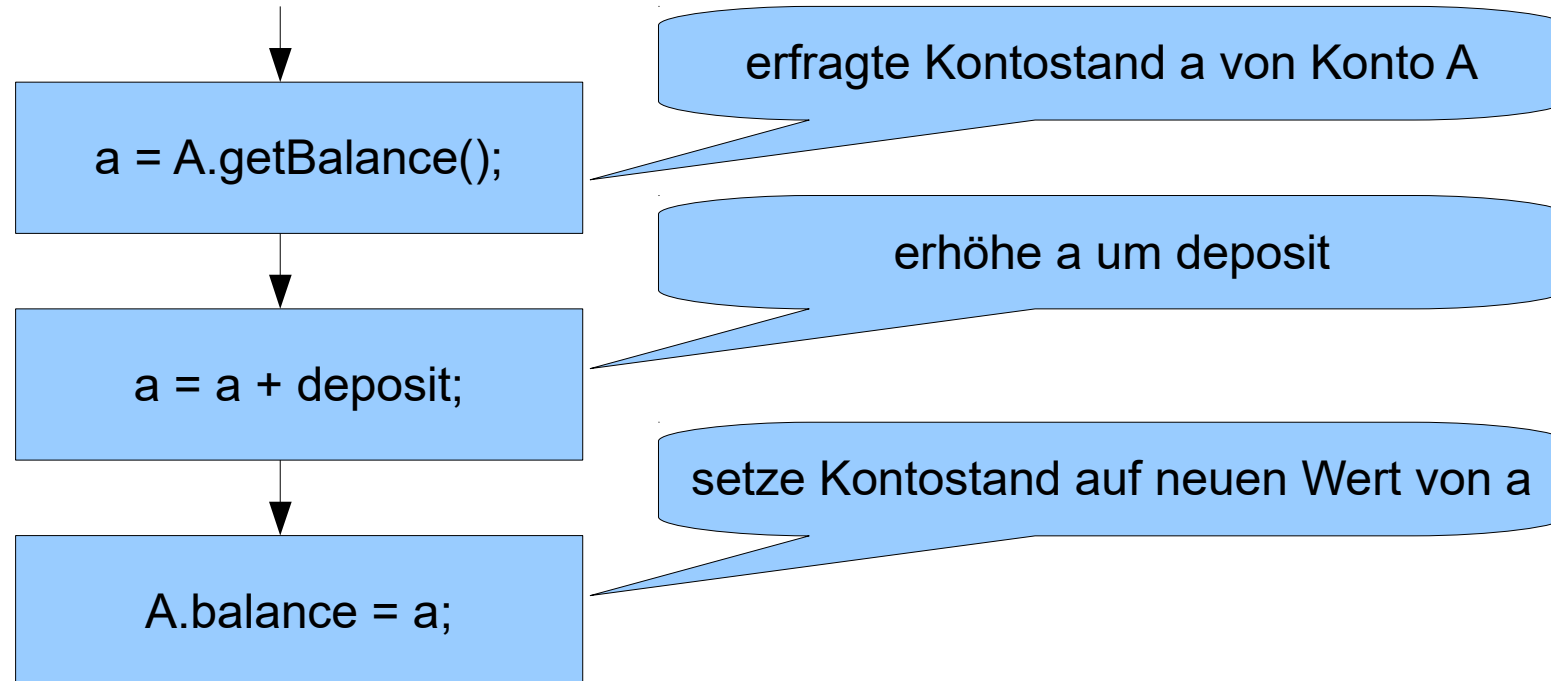
- Modellgetriebene SW-Entwicklung
  - Nebenläufige Programmierung
- Qualitätsmanagement
- Testen



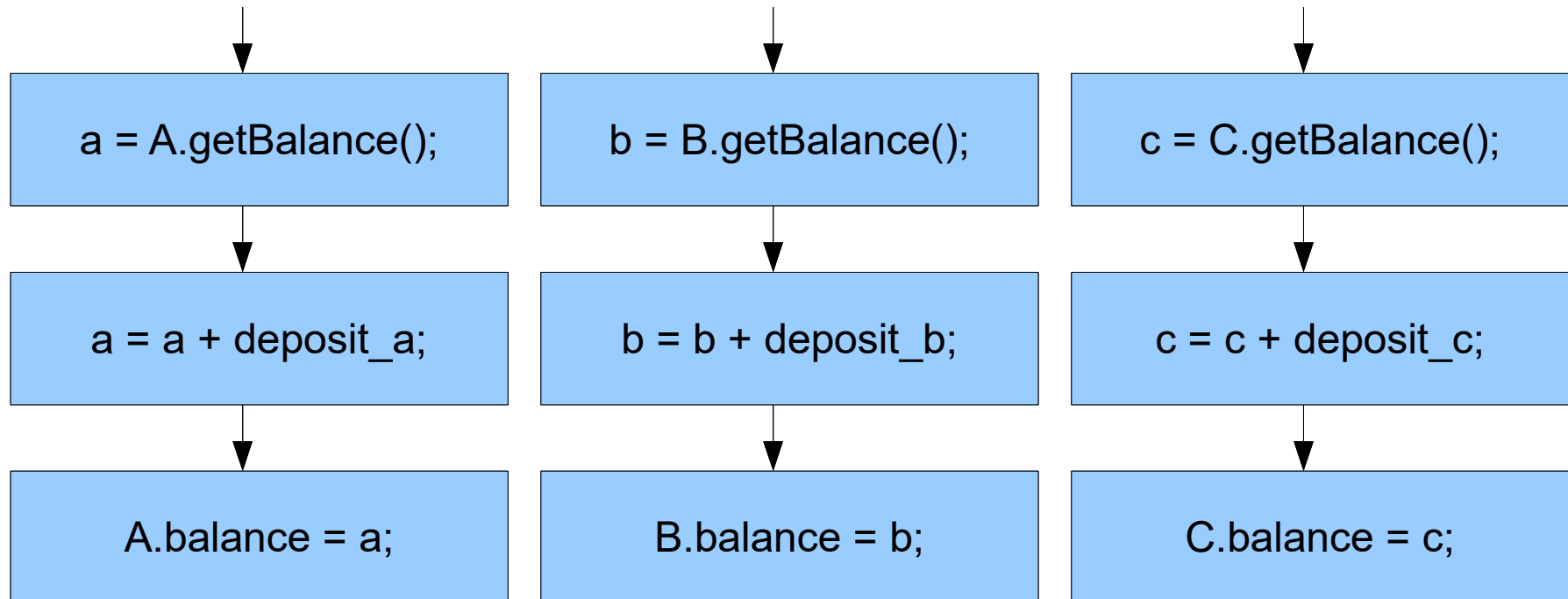
- Bewusstsein für die Schwierigkeiten und Anforderungen für und an nebenläufiger Programmierung.
- Verschiedene Konzepte und Methoden zur nebenläufiger Programmierung unterscheiden können.
- Ausgewählte Methoden der nebenläufigen Programmierung anwenden und kombinieren können.

- Ein **System** ist von seiner Umgebung abgegrenzte Anordnung von Komponenten.
- Können in einem System mehrere Aktivitäten gleichzeitig stattfinden, spricht man von einem **nebenläufigen (parallel ablaufenden) System**.
- Ist das System aus räumlich verteilten Komponenten aufgebaut, spricht man von einem **verteilten System**.
- In einem **nebenläufigen System** laufen mehrere Kontrollflüsse gleichzeitig ab und können miteinander kommunizieren.

Der Stand eines Konto wird abgefragt, eine Summe „deposit“ eingezahlt, die Kontostandartvariable um einen Betrag „deposit“ erhöht und der neue Wert als neuer Kontostand eingetragen:



Einzahlungen können auch parallel verlaufen:

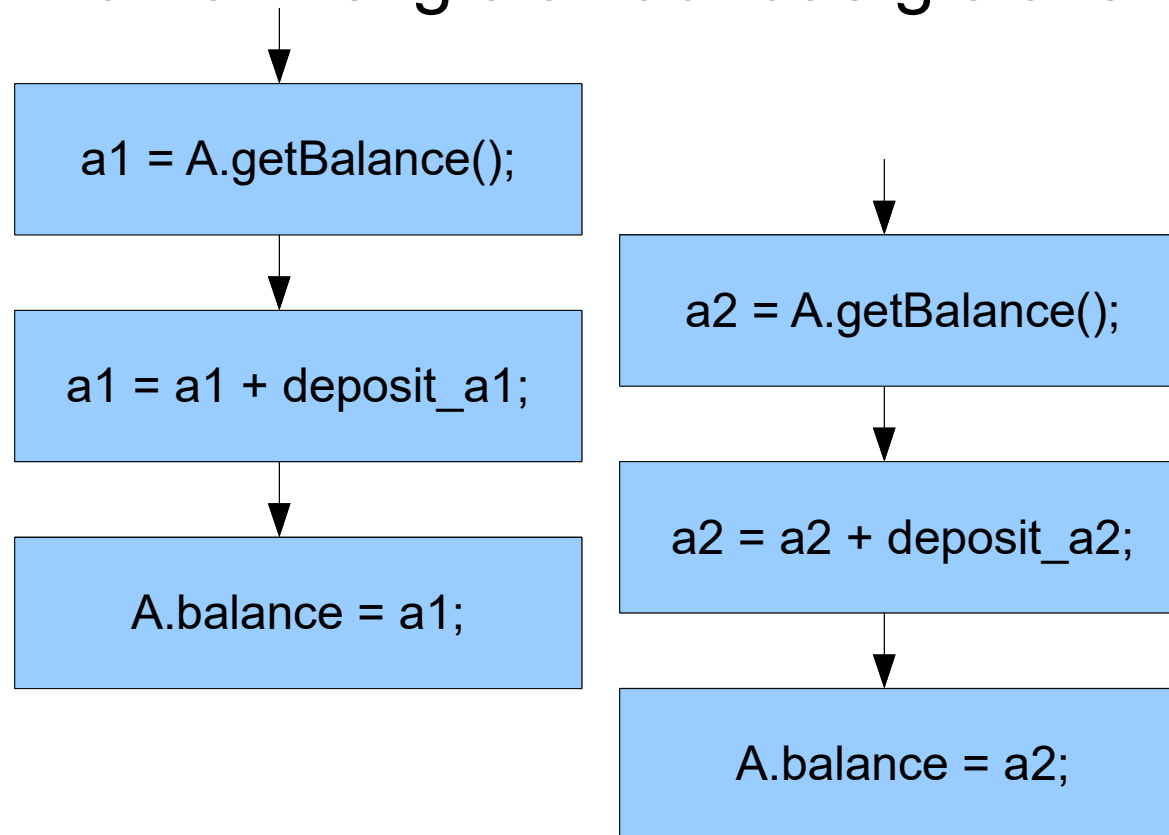


Innerhalb eines Computers wird dies „multi-threading“ genannt. Ein Thread kann eine Aufgabe unabhängig von anderen Threads erfüllen.

- Aus Betriebssystemensicht wird ein **Prozess** verstanden als **abstrakte Maschine, die eine Berechnung ausführt**.
- Ein **Thread** ist ein „**leichtgewichtiger Prozess**“, der einen eigenen Stack, aber keine eigenen Heap besitzt.
- Er ist Teil eines Programms, das unabhängig von anderen Teilen des Programms ausgeführt werden kann, und repräsentiert eine einzige Sequenz von Anweisungen, d.h. einen **sequentiellen Kontrollfluss**, der **nebenläufig zu anderen Threads** ausgeführt werden kann und der Daten des Heaps (Instanzvariablen, statische Variablen und Elemente von Reihenungen) **gemeinsam mit anderen Threads** benutzt.

- Lokale Variable, formale Parameter und Parameter von Ausnahmebehandlern:
  - sind lokal für jeden Thread;
  - haben keinen Einfluss auf andere Threads.
- Instanzvariable, statische Variable, Elemente von Reihungen:
  - residieren in einem globalen Speicher.
  - Änderungen können die Werte anderer Threads beeinflussen.

Zwei Kunden zahlen zeitgleich auf das gleiche Konto ein:



Erster Eintrag verloren → „Race Condition“, Interferenz zwischen Threads

- Eine **Race Condition** ist eine Situation, in der zwei oder mehr Threads gemeinsame Daten schreiben oder lesen und das Endergebnis vom Scheduling der Threads abhängt.
- Race Conditions sind gefürchtet, da sie zu **schwer vorhersagbarem und schwer reproduzierbarem Programmverhalten** führen.
- Eine Race Condition manifestiert sich oft nur sehr selten (etwa 1 Mal pro 1000 Programmabläufe) und ist daher **durch Testen kaum aufzuspüren**.

- Interaktion zwischen Threads kann zu unerwünschtem Verhalten führen.
- Zwei Arten von Eigenschaften nebenläufiger Programme:
- **Sicherheit** = Eigenschaft, dass nie etwas Unerwünschtes geschieht
- **Lebendigkeit** = Eigenschaft, dass immer irgendetwas geschieht
- **Sicherheitsfehler** führen zu unerwünschtem Laufzeitverhalten
- **Lebendigkeitsprobleme** führen dazu, dass das Programm anhält, obwohl noch nicht alle Threads terminiert haben.

Sicherheitsprobleme basieren häufig auf

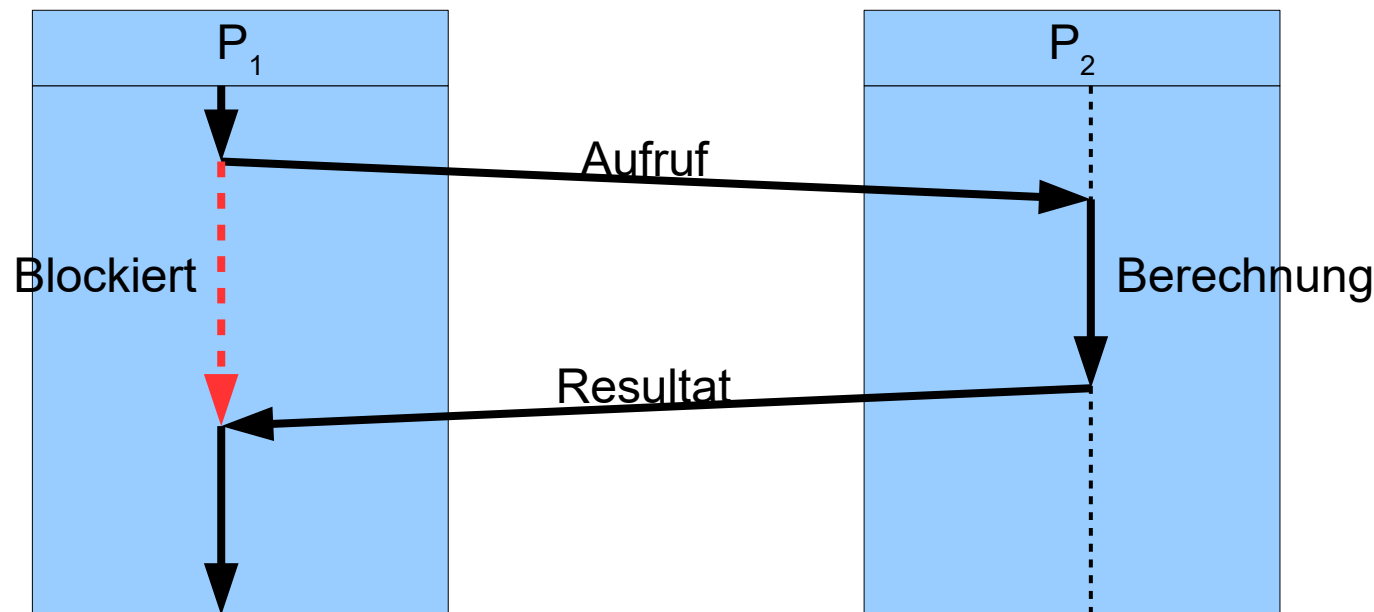
- **Lese / Schreibkonflikten**

- Der Zugriff eines lesenden Kunden auf ein Objekt während eines noch nicht beendeten Schreibvorgangs kann zum lesen inkorrektur Daten führen
- Beispiel: Lesen einer Reihung während alle Feldelemente verdoppelt werden

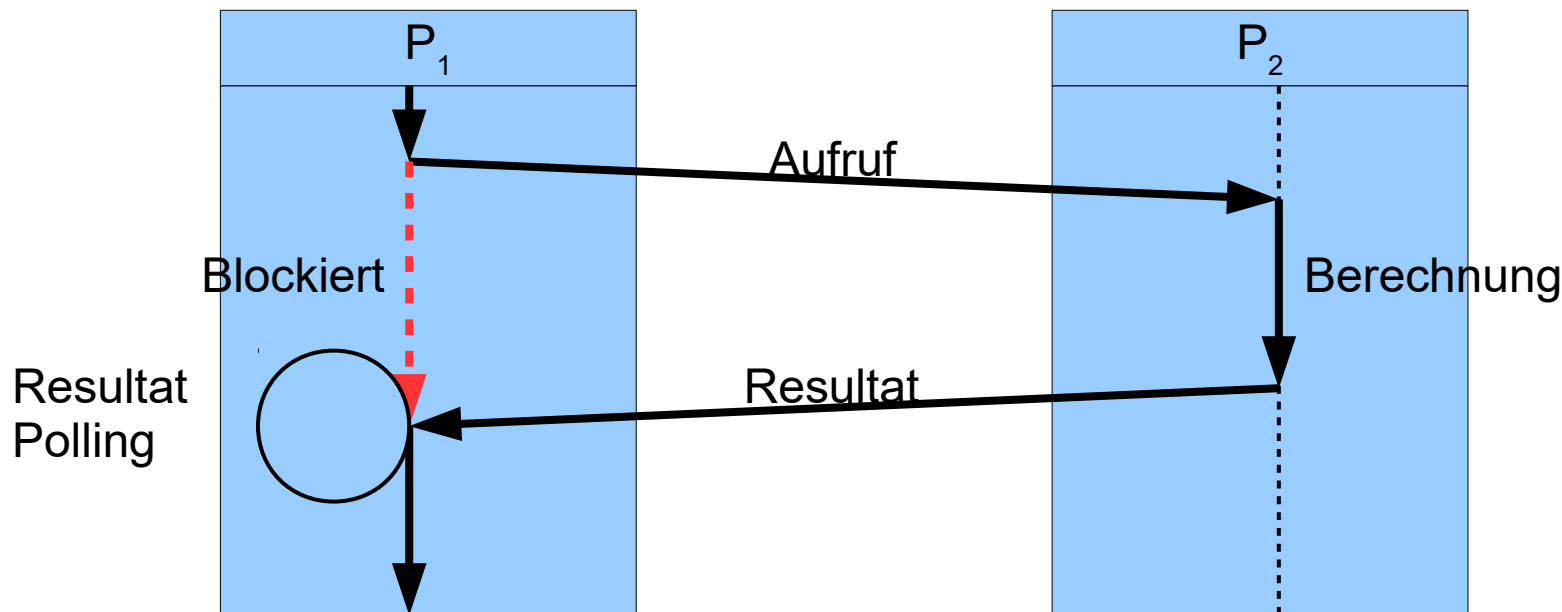
- **Schreib/Schreibkonflikte**

- Inkonsistente Zuweisung an Variablen durch nebenläufiges Ausführen von zustandsändernden Methoden

- Prozess  $P_1$  ruft Service von  $P_2$  synchron auf
- $P_2$  benötigt Zeit um Resultat bereitzustellen
- Was macht  $P_1$  in der Zwischenzeit?



- Prozess  $P_1$  ruft Service von  $P_2$  asynchron auf
- $P_2$  benötigt Zeit um Resultat bereitzustellen
- $P_1$  arbeitet weiter und muss  $P_2$  Resultat pollen



- **Proxy** für **Ergebnis**, das noch nicht bekannt ist
- Future = Rückgabewert einer asynchronen Funktion (promise)
- Zugriff möglich, wenn Berechnung abgeschlossen
- Futures werden als Argumente an andere Prozeduraufrufe **weitergereicht**
- Auswertung beginnt, wenn Ergebnis des Futures vorhanden
- Konzept: Future hat get- oder join-Funktion, die so lange **blockiert**, bis Ergebnis vorliegt, und dieses dann zurückliefert

```
interface ArchiveSearcher { String search(String
target); }
class App {
    ExecutorService executor = ...
    ArchiveSearcher searcher = ...
    void showSearch(final String target)
        throws InterruptedException {
        Future<String> future
        = executor.submit(new Callable<String>() {
            public String call() {
                return searcher.search(target);
            }
        });
        displayOtherThings();//do other things
        try {
            displayText(future.get()); // use future
        } catch (ExecutionException ex){cleanup(); return;}
    }
}
```

Quelle: Oracle Java™ Platform Standard Ed. 8 Documentation

- Nebenläufige Programmierung besitzt eigene Fehlerklassen, Eigenschaften und Methoden
- Futures und Promises sind Möglichkeit nebenläufige Programme zu entwickeln

## EMF und Meta-Programmierung